

CHAT4SEED: Semantic-Awareness Highly Structured Seed Generation for Fuzzing

Jiarui Chen, Bin Zhang*, Jiongyi Chen, Runhao Li, and Chaojing Tang

National University of Defense Technology, Changsha, China

rjc0081@nudt.edu.cn, binzh4ng@hotmail.com, chenjiyongyi@nudt.edu.cn, lirinuhao@nudt.edu.cn, tangchaojing@nudt.edu.cn

*corresponding author

Abstract—Fuzzing, one of the most popular methods for enhancing software security and quality, relies heavily on the quality of its initial seed corpus to effectively uncover vulnerabilities. Traditional methods for generating initial seed corpora, whether crawl-based or generation-based, struggle with programs that handle highly structured formats with complex semantics, leading to low testing coverage and reduced fuzzer effectiveness. Although researchers have proposed leveraging Large Language Models (LLMs) to create high-quality seed corpora, current approaches are limited to text-based seed files, such as JavaScript code. To address the limitations, we propose CHAT4SEED, a novel approach that extends the capabilities of LLMs to produce not only text but also highly-structured binary seed files. CHAT4SEED leverages LLMs to extract and interpret the semantic constraints embedded within format-specific branches of programs. While existing approaches focus solely on generating text-based seeds, CHAT4SEED goes further by utilizing LLMs to generate functional library invocation code to produce binary seeds. It also employs binary-level manipulation to handle unsupported corner cases, achieving robust seed generation for both text and binary formats. Our experiments and evaluations on 12 real-world programs demonstrate that after 48 hours of fuzz testing using AFL++, the seed corpus generated by CHAT4SEED achieves an average increase in coverage of 28.78% compared to traditional crawl-based methods and 39.98% compared to generation-based methods. Additionally, it facilitates the discovery of 84.24% and 392.7% more crashes than seed corpus generated by traditional approaches, respectively, underscoring the potential of CHAT4SEED to enhance the efficacy of fuzzing.

Keywords—software testing; fuzzing; seed generation; large language models

1. INTRODUCTION

Fuzzing has become one of the most popular methods for identifying program flaws and enhancing software quality [1], [2], [3]. Beginning with an initial seed corpus, the fuzzing engine mutates numerous inputs and feeds them to the target program to uncover potential flaws. Empirical studies underscore the substantial impact of seed quality on the ability of the fuzzer to detect vulnerabilities. Klees et al. emphasis that “it is clear that a fuzzer’s performance on the same program can be very different depending on what seed is used” [4]. As demonstrated in [5], providing AFL++ with an initial seed corpus collected from public significantly enhances the testing coverage, increasing it to approximately 60% compared to

using an uninformed seed. This finding is corroborated by the practices of widely deployed fuzzers like OSS-Fuzz [3], which emphasize the importance of initial seed quality in optimizing the fuzz testing process. Therefore, the generation of high-quality initial seed corpus is an essential research issue.

Current methods of initial seed corpus generation for fuzzing can be divided into two types, i.e., crawl-based and generation-based. Although crawl-based seeds are plentiful and easy to obtain, their initial coverage of target programs is limited due to the absence of a direct association with these programs. A data-driven seed generation approach utilizes a collection of artificially constructed and self-learned grammar rules to produce new seeds through mutation (e.g., Skyfile [6]). While it adheres to the format constraints of the target program’s input files, it does not utilize the internal constraint information of the program. Another method for seed generation involves the utilization of symbolic execution (e.g., SymCC [7]), which employs constraint solvers like z3 [8] to generate inputs for branches.

While current methods for generating initial seed corpus offer certain advantages, they remain less suitable for programs that handle highly structured inputs, such as images and executables. These programs often contain a significant number of *format-specific branches* (FSB) that perform various checks on specific fields within the input format. For instance, an image parser may determine whether a JPEG image was captured at a particular location by examining its GPS metadata field. To cover this FSB, one needs to construct a JPEG image with the specific GPS coordinates. Generating such an image is challenging for existing seed corpus generation methods. Either mutating sample files according to learned rules or solving branches as arithmetic constraints, may still struggle to generate the needed GPS metadata. Although previous studies have explored the use of Large Language Models (LLMs) for understanding program source code and generating corresponding test cases, these efforts have been limited to text-based files like JavaScript code (e.g., CovRL-Fuzz [9]). In this paper, we introduce CHAT4SEED, an innovative framework that leverages LLMs to generate high-quality initial seed corpora, capable of handling both text-based and binary inputs for fuzzing programs with highly structured inputs. **Our core insight is that an experienced programming expert with a comprehensive understanding of the file format can quickly utilize third-party libraries to create input files that cover a given branch once the branch’s constraints are understood.** Although LLMs are not designed for creating initial seed corpus, they hold significant

potential for generating desired seed files. This is due to their extensive pre-training on diverse datasets, which equips them with two essential capabilities: understanding natural language and code, as well as the ability to write code. Based on this insight, CHAT4SEED enables the flexible utilization of LLMs to semantically analyze the constraints of FSBs, thus clarifying the requirements for target inputs that can satisfy these constraints. After understanding the constraints, unlike traditional mutating or solving methods, CHAT4SEED drives the LLM to generate text seeds directly or generate code to cover the constraints by invoking third-party Python libraries that handle the target file format, thereby producing the required binary inputs. By iteratively generating input files for FSBs identified in the target program, CHAT4SEED ultimately create a high-quality initial seed corpus with better initial code coverage. This approach facilitates the creation of initial seed corpus for programs that process highly structured binary files such as images and executables, as well as raw text data, including XML and XSL.

The main contributions of this paper are listed as follows:

- We proposed a novel method that generates initial seed corpus for fuzzing by addressing branch constraints from a semantic perspective. Unlike traditional approaches, our approach involves understanding the branch constraints within the code, which allows us to produce Python code to construct inputs that can cover the branch. This method effectively tackles the challenge of achieving coverage for FSBs in programs that process highly structured inputs, resulting in a remarkable improvement on testing coverage for fuzzing.
- We devised an automatic framework, CHAT4SEED, that generates seed files for programs that process highly structured binary and text inputs using LLM. By seamlessly integrating into a meticulously constructed program analysis workflow, and combining with knowledge databases as well as binary file parsing tools, the LLM can identify and comprehend constraint information within the FSBs, and then generate seed files that satisfy these requisitions by either producing library invocation code or manipulating the input directly at the byte level.
- We implemented a prototype of CHAT4SEED and evaluated it on 12 real-world programs, encompassing input types such as images, executables, and hypertext. Experimental results demonstrate that the seed corpus generated by CHAT4SEED achieved an average improvement of 28.78% and 39.98% in the fuzzer's test coverage compared to traditional crawl-based and generation-based methods, and discovers 84.24% and 392.7% more crashes, respectively, thereby demonstrating the potential of the LLM in the field of initial seed corpus generation.

2. MOTIVATING EXAMPLE

Listing. 1 illustrates an example code that checks the GPS information from the EXIF metadata. Function `process_EXIF` calls `ProcessExifDir` at line 8 to handle each EXIF directory entry. If an entry is tagged with GPS

Listing 1 Example Code that checks the GPS information from the EXIF metadata of an image.

```

1 int process_EXIF(...) {
2     ...
3     if (memcmp(ExifSection + 8, "II", 2) == 0) { ... }
4     else if (memcmp(ExifSection + 8, "MM", 2) == 0) { ... }
5     else return 0;
6     if (Get16u(ExifSection + 10) != 0x2a) return 0;
7     if (FirstOffset < 8 || FirstOffset > 16) return 0;
8     ProcessExifDir(...);
9 }
10
11 static void ProcessExifDir(...) {
12     ...
13     if (Components > 0x10000) continue;
14     if ((Format - 1) >= NUM_FORMATS) continue;
15     if ((unsigned)Components > 0x10000) continue;
16     switch (Tag) {
17     case TAG_GPSINFO:
18         unsigned char *SubdirStart = OffsetBase + Get32u(ValuePtr);
19         if (SubdirStart < OffsetBase) { ErrNonfatal("Illegal GPS", 0, 0); }
20         else { ProcessGpsInfo(...); }
21         ...
22     }
23 }
24
25 void ProcessGpsInfo(...) {
26     ...
27     for (de = 0; de < NumDirEntries; de++) {
28         if (Components > 0x10000) continue;
29         if ((Format - 1) >= NUM_FORMATS) continue;
30         switch (Tag) {
31         case TAG_GPS_LAT:
32             for (a=0;a<3;a++){
33                 int den = Get32u(ValuePtr+4+a*ComponentSize);
34                 int digits = 0;
35                 while (den > 1 && digits <= 6){
36                     den = den / 10;
37                     digits += 1;
38                 }
39                 if (digits > 6) digits = 6;
40                 FmtString[1+a*7] = (char)('2'+digits+(digits ? 1 : 0));
41                 FmtString[3+a*7] = (char)('0'+digits);
42                 Values[a] = ConvertAnyFormat(ValuePtr+a*ComponentSize, Format);
43             }
44             snprintf(TempString, sizeof(TempString), ...);
45             ImageInfo.GpsLat = TempString;
46             break;
47         case ... :
48             ...
49         if (ImageInfo.GpsLat == "40°26'0.0'' N") { // <---- target branch here
50             ...
51         }
52     }
53 }

```

information (line 17), it then invokes `ProcessGpsInfo` at line 20 to parse detailed information (i.e., longitude and latitude) to determine if it is captured at $45^{\circ}26'0''N$.

The branch at line 49 is a typical format-specific branch that checks on the GPS information field within the input image. Referring to the detailed structure of the EXIF data within an image file as illustrated in Fig. 1, to cover this branch, the input image file must satisfy the criteria that highlighted in red as following:

- A correct TIFF Header structure, indicating that this data segment stores the EXIF data as an APP1 data segment.
- Byte alignment must match with all subsequent data.
- A correct IFD0 Offset, pointing to the beginning of the IFD0 data segment.
- A Directory Entry with a Tag value of 0x8825, where the Data segment stores the offset to the GPS IFD.
- A valid GPS Directory Entry within the GPS IFD.
- The GPS latitude equals to " $45^{\circ}26'0''N$ ".

Crawling an image that meets the above criteria from online sources is almost impossible. Additionally, current generating-based methods face challenges in constructing the image that covers this branch. The process of randomly mutating from learned rules may lead to a considerable long time to obtain the desired latitude value. Furthermore, the complex data conversion code between lines 31 and 43 poses difficulties for constraint solvers to obtain a valid solution.

While traditional methods may find it challenging to address such format-specific branch when processing highly structured

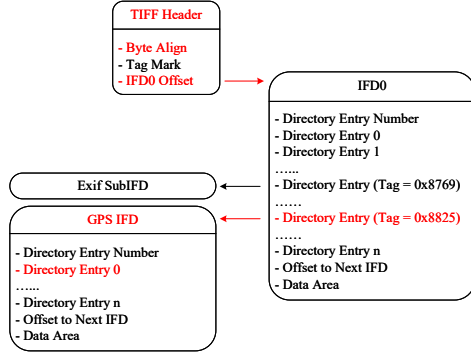


Figure 1: EXIF Structure.

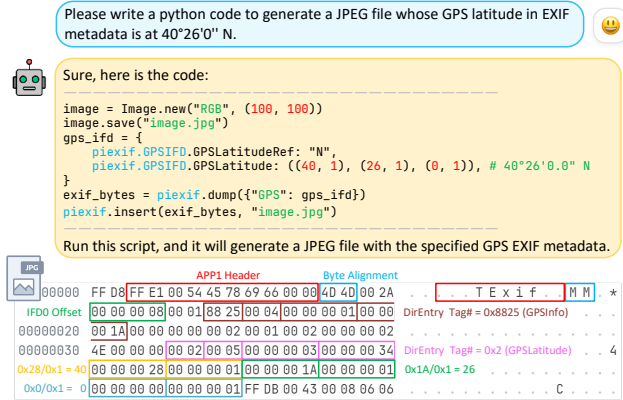


Figure 2: LLM successfully generates a valid seed that covers the format-specific branch at line 49 in Listing. 1.

input types, there is potential to leverage LLM to generate input files to cover such branch. LLM exhibits fundamental code understanding & writing capabilities, and they are supported by extensive training data encompassing a wealth of information on file formats. As shown in Fig 2, when given the semantics implied in the branch condition, LLM successfully generates a valid seed that satisfies the target branch condition through invoking *piexif* library and configuring the relevant parameters.

While leveraging LLMs to generate the desired input file from the semantic perspective holds great potential, applying LLM to automatically generate an initial corpus for fuzzing programs that process highly structured inputs still faces the following challenges:

- **How to identify and comprehend the constraints related to a format specific branch by LLM?** Despite LLMs possess fundamental capability for code comprehension, they struggle to directly grasp the intricate constraints present in real-world programs. The constraints of a given branch may depend on preceding statements, complicating the extraction of the FSB's constraints. Providing the LLM with only local information from the current branch statement can result in invalid seed generation. Additionally, traditional approaches, such as symbolic execution, represent constraints as bit-vector expressions, which make

convenient for constraint solvers like Z3 [8]. However, LLMs are incapable of comprehending byte-oriented data; it is necessary to explore a natural language form that LLMs can understand to represent constraints.

- **How to generate functional library invocation code to cover an FSB based on its constraints by LLM?** Although the base models such as ChatGPT exhibit the ability to generate code, they still encounter notable challenges in producing functional library invocation code necessary for covering an FSB. Various factors, such as the diverse nature of file formats, the varying coding styles of libraries designed for format processing, and the limited availability of sample code, lead to the potential errors in the library invocation code generated by LLMs. These errors hinder the generation of desired input data that satisfies the constraints of the FSB.
- **How to generate corner cases for branches that current libraries do not support?** Real-world programs contain a significant amount of branches that handle malformed input files, such as checking whether the image width is negative. Such malformed inputs often serve as seeds with greater potential for triggering program failures. However, most libraries discard these exceptional conditions, as they are primarily designed to generate files in the correct format. When attempting to construct input data that meets the constraints of these corner cases, the formatting library raises errors, thereby hindering the generation of the desired input data and resulting in low bug discovery efficiency.

3. SYSTEM DESIGN

The main framework of CHAT4SEED is shown in Fig. 3. Initially, drawing upon the work of Asmita et al. [10], we employ prompts to instruct the LLM to generate an startup seed set. Based on this seed set, we conduct code coverage testing and perform a static analysis of the software's source code to identify key branches that yield the most significant coverage improvement once covered. Subsequently, by employing meticulously crafted prompt engineering, we direct the LLM to filter out the format-specific branches (FSBs) from those key branches, and extracts their constraints from the identified relevant code context. The LLM can directly generate text seeds after understanding the constraints, while for binary seeds, we utilize the LLM to generate functional library invocation code (LIC) aimed at covering these identified FSBs, supported by a knowledge database. For branches that address corner cases not supported by current libraries, we propose a binary-level input manipulation method that directly modifies the bytes of a normal input to meet the requirements of these corner cases. The newly generated seeds are incorporated into the seed set, followed by the execution of incremental code coverage tests to initiate the subsequent iteration. The process of CHAT4SEED terminates its iteration once all reachable FSBs have been analyzed, at which point it outputs the final seed corpus.

The following sections provide a detailed exposition of CHAT4SEED framework.

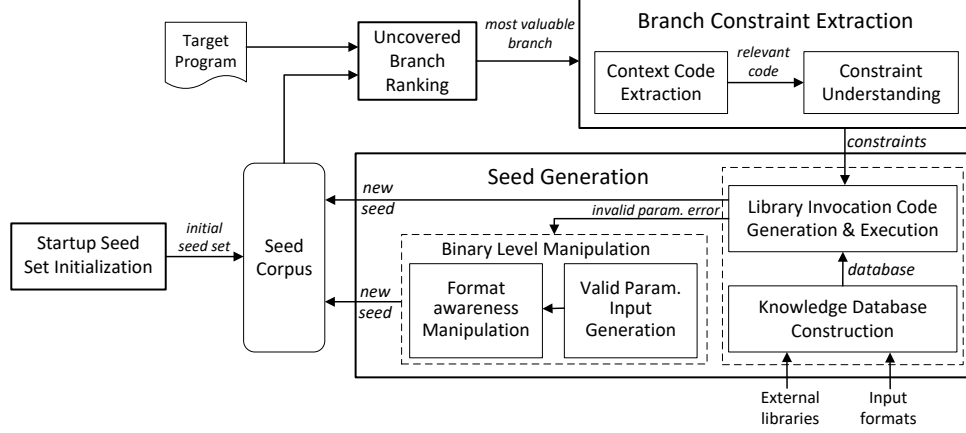


Figure 3: Overview of CHAT4SEED

Algorithm 1 Weight calculation algorithm of a branch.

Require: All basic blocks $\mathcal{B}$ within the branch's body.

Ensure: Branch weight w .

```

1:  $w \leftarrow 0$ 
2: Function list  $\mathcal{F} \leftarrow \emptyset$ 
3: for  $bb \in \mathcal{B}$  do
4:   if not covered( $bb$ ) then
5:      $w \leftarrow w + 1$ 
6:   if contains_call( $bb$ ) then
7:      $f \leftarrow \text{get\_call\_target}(bb)$ 
8:     if  $f \notin \mathcal{F}$  then
9:        $\mathcal{B}_f \leftarrow \text{get\_BBs}(f)$ 
10:       $w' \leftarrow \text{Weight}(\mathcal{B}_f)$ 
11:       $w \leftarrow w + w'$ 
12:       $\mathcal{F} \leftarrow \mathcal{F} \cup f$ 
13:     end if
14:   end if
15: end for
16: return  $w$ 

```

3.1 Priority of Branch for Seed Generation

The number of branches in real-world programs can be substantial, and each invocation of the LLM incurs significant time and cost. Generating all possible branches individually would therefore be impractical and resource-intensive. To address this challenge, we developed a prioritization scheme that assigns a weight factor to each branch, indicating its potential to enhance the testing coverage of the fuzzer. Branches with higher weight factors are deemed more impactful in improving coverage and should be prioritized accordingly.

The weight calculation algorithm is shown in Algorithm 1. For each basic block bb within the body of the given branch, if it has not been covered by the seed corpus generated so far, we increase the branch's weight w by 1 (line 4-5). Additionally, if bb contains a call statement and the target function has not been previously seen in this calculation process (line 6-8), we compute the weight w' of this function by applying this algorithm to all basic blocks within the function (line 9-10). Subsequently, we update the weight of the given branch by adding w' , the weight of this function. Summing the weights of all basic blocks within the body of the branch yields the

final weight of the branch.

Among various types of branches, the *switch-case* statement holds a unique position where all *case* statements within it share the same *switch* statement, which means they have highly similar constraints. To improve the efficiency of our analysis, instead of treating each *case* individually, we consolidate all *case* branches under a single composite *switch* branch. This approach helps avoid redundant invocations of LLM. Based on this observation, we define the weight of a *switch-case* branch w_{sc} as following:

$$w_{sc} = \sum_{i=1}^N w_{c_i} \quad (1)$$

, where N is the number of *case* statements within the body of a *switch-case*, w_{c_i} denotes the weight of branch produced by each *case* statement.

Once a newly generated seed file is added into the seed corpus, the weight of all uncovered branches are re-calculated.

3.2 Format-Specific Branch Constraint Comprehension

Given a branch with the highest weight factor, CHAT4SEED extracts the relevant context code associated with this branch. Based on the relevant context code, CHAT4SEED then determines whether it qualifies as an FSB. For an identified FSB, CHAT4SEED extracts its constraint information and presents it in a template that can be understood by the LLM for subsequent seed generation.

3.2.1 Relevant Context Analysis

As previously mentioned, the constraints associated with a branch can depend on earlier statements. Limiting the prompt to only the current branch's information may lead to erroneous outcomes. Hence, to ensure the correct extraction of constraints, we employ backward slicing on the static value flow graph of variables within the target branch to identify all pertinent preceding statements.

This approach allows us to extract additional pertinent code, thereby facilitating a more comprehensive analysis by the

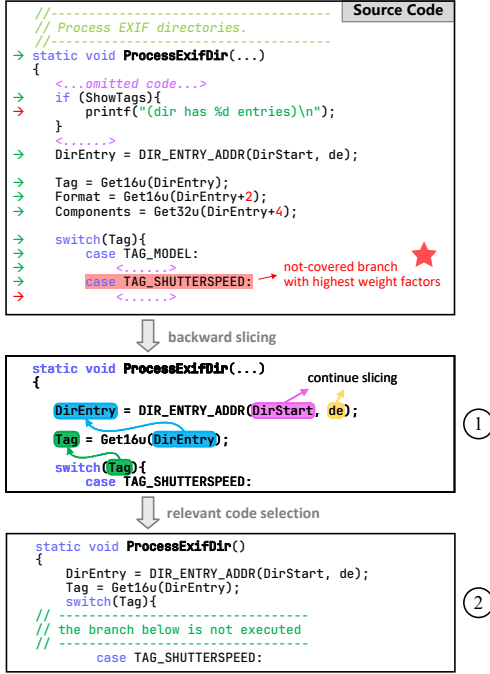


Figure 4: Example of backward slicing and relevant code selection.

LLM. Fig. 4 illustrates the process of conducting a code backward slicing on the conditional variables depicted in the FSB. For instance, in the source code block, the not-covered FSB with highest weight factors is `case TAG_SHUTTERSPEED`. Backward tracing of the `tag` conditional variable reveals that it is the return value of the `Get16u` function, which takes its input from the `DirEntry` parameter. This parameter originates from the `DIR_ENTRY_ADDR` function, and the entry parameter of this function can be further traced, shown as block ① in Fig. 4.

The final extract relevant code is shown as block ② in Fig. 4. Note that we restrict the backward slicing steps to a finite number to ensure that the total size of the relevant code does not exceed the token limits of the LLM.

3.2.2 FSB Constraint Comprehension by Prompting

CHAT4SEED employs the LLM to assess whether a branch qualifies as an FSB. The prompt template used for this purpose is illustrated in Fig. 5 (a), which incorporates key inputs such as information about the program and the extracted relevant code context. Furthermore, to enhance the LLM’s judgment capabilities, we have included explicit few-shot examples within the prompt to improve the accuracy of FSB identification.

The prompt template for FSB constraint extraction is illustrated in Fig. 5 (b). To assist the LLM in focusing on the task of branch constraint comprehension, we provided a task-related background introduction in the prompt, clarifying that the primary task at hand is program testing, which requires the generation of files that can meet the branch constraints to improve testing coverage. We require the LLM to comprehend

the constraints, and identify the field names and corresponding values in the file format that meet the specified constraints. Since the extracted constraints are subsequently provided to the LLM for seed file generation, we developed a response template that is understandable by the LLM to express these branch constraints. The response template is as follows: “*This constraint is associated with the $\langle \text{field name} \rangle$ in the data file; hence, a file containing $\langle \text{field value} \rangle$ is required to satisfy this constraint*”. This template clearly specifies which field of the input format is relevant and what value should be present in that field within the input file.

3.3 Library Invocation Code Generation and Execution

Upon obtaining the constraint information from the FSB, for text-based file formats, CHAT4SEED requires the LLM to directly provide it within the response [10]. For binary formats, CHAT4SEED directs the LLM to write the functional Library Invocation Code (LIC), which is designed to produce input files that meet the provided branch constraints.

3.3.1 LIC Generation Prompting

Although LLMs have shown impressive capabilities in code generation for general tasks, such as generating web page code, challenges persist when utilizing LLMs to produce functional LIC. These challenges stem from the limited availability of template code for invoking external libraries and the complexity and diversity of input file formats. In addition to the common syntax error, Fig. 6 lists the typical errors when directly using LLM to produce LIC, which can be grouped as followings:

- **Invoking with incorrect attribute names:** As shown in Fig. 6 (A), the LLM erroneously sets the GPS IFD attribute name to “GPSTAGS” instead of the correct value “GPSIFD”.
- **Invoking with incorrect parameter formats:** As shown in Fig. 6 (B), the data format for GPS longitude should be a set of rational numbers, with each rational number consisting of two integers representing the numerator and the denominator. However, the LLM incorrectly passed the parameter value by directly setting the three coordinate values in the dictionary without converting them into the rational number format.
- **Invoking non-exist methods:** As illustrated in Fig. 6 (C), the LLM erroneously invokes a non-existent method, “piexif.save,” instead of the correct method, “piexif.insert,” to embed the EXIF metadata into a jpeg file.

We categorize the aforementioned error types as attribute misalignment errors. To address this issue, we have developed a knowledge database, which includes detailed information about external libraries and input formats, to augment the LIC generation capability of LLM through Retrieval-Augmented Generation (RAG) [11]. The database comprises three types of knowledge: library invoking information, binary structure of format field, and semantic of format field. All knowledge data is sourced from publicly available resources, including library documentation and general information about various formats.

(a) FSB identification prompt template	(b) FSB constraint extraction prompt template
<pre> # Definition 'Format Specific Branch' is a branch related to one or more fields within the file format. ## Positive Example (answer YES) Branch: 'case TAG_MODEL' Reason: Can be explicitly matched to the EXIF field. ## Negative Example (answer NO) Branch: 'i++' Reason: Simple and basic control flow code. Branch: 'ShowTags' Reason: Potentially related to the file format, but with indeterminate correspondence. Branch: 'PatCopy[a] == 'x'' Reason: Uncertain relevance to the file format. Branch: '!strcmp(arg, "-ar")' Reason: Can be definitively ascertained as unrelated to the file format. # Task Determine whether the target branch in code context is a 'Format Specific Branch'. # Information about the program { prog_info } # Branch relevant code context { relevant_code_context } ----- Now answer YES or NO and give me your reason. </pre>	<pre> # Background I have already established an initial test suite that has generated a certain degree of code coverage within the program. Currently, I'm encountering a branch where the existing test cases have not met the branch constraint, resulting in the subsequent code following the branch not being executed. To enhance the code coverage, it is necessary to generate a new test case that is designed to fulfill the specific branch constraint, thereby executing the code that follows this branch. # Information about the program { prog_info } # Branch relevant code context { relevant_code_context } # Goal By comprehending the constraints within the target branch, identify the field names and their corresponding values that are associated with the file format. # Answer Format This constraint is associated with <field name> in the file; hence, a file containing <field value> is required to satisfy this constraint. ----- Now give me the the answer following the 'Answer Format'. </pre>

Figure 5: Prompt template for FSB identification and constraint extraction.

	Generated by LLM	Correct Code
A →	<pre>gps_ifd = { piexif.GPSIFD.GPSLatitudeRef: 'N', piexif.GPSIFD.GPSLatitude: (latitude[0], latitude[1], latitude[2]), piexif.GPSIFD.GPSLongitudeRef: 'W', piexif.GPSIFD.GPSLongitude: (longitude[0], longitude[1], longitude[2]), }</pre>	<pre>gps_ifd = { piexif.GPSIFD.GPSLatitudeRef: 'N', piexif.GPSIFD.GPSLatitude: (latitude[0], latitude[1], latitude[2]), piexif.GPSIFD.GPSLongitudeRef: 'W', piexif.GPSIFD.GPSLongitude: (longitude[0], longitude[1], longitude[2]), }</pre>
B →	<pre>exif_dict = {"GPS": gps_ifd} exif_bytes = piexif.dump(exif_dict)</pre>	<pre>exif_dict = {"GPS": gps_ifd} exif_bytes = piexif.dump(exif_dict)</pre>
C →	<pre>piexif.save(exif_bytes, 'image_with_exif.jpg')</pre>	<pre>piexif.insert(exif_bytes, 'image_with_exif.jpg')</pre>

Figure 6: The incorrect Library Invocation Code (LIC) generated by the LLM, highlighted in red, is compared with the correct code, highlighted in green.

We downloaded these resources from websites and utilized the text-splitters tool provided by Langchain [12] to segment the documents based on their formatting. The resulting text blocks are then stored as vectors in a vector database.

When provided with the branch constraint, CHAT4SEED constructs the LIC generation prompt as listed in Fig. 7. It first converts the constraint into vector, and then employs vector similarity to extract relevant information (i.e., *lib_info* and *file_format_desc* which is combined with binary and semantic description of the field) from the knowledge database. After that, this extracted information is embedded into the prompt along with the branch constraint *branch_cons_desc* and a summary of the target program *prog_info*. At last, the final prompt is submitted to the LLM to produce LICs.

3.3.2 Execution Exception Handling

Despite the advantages provided by the knowledge database, the LLM may still occasionally produce LICs that contain errors and fail to execute. In contrast to those attribute misalignment errors, which can be identified through static code analysis, there are certain errors that can only be detected during code execution, such as unsupported parameters failure.

For instance, the *ImageWidth* parameter needs to be set to -1 to satisfy branch conditions, but the *piexif* library does not allow setting *ImageWidth* to a negative value. To address this issue, we introduced an execution exception handling phase designed to automatically identify and resolve exceptions that occur during the execution of LICs.

When an exception raises during the execution of LICs, we submit the error message and call stack reported by the Python interpreter to the LLM, and leverage the Chain-of-Thought (CoT) [13] strategy to guide the LLM to identify the root causes step by step. We first require the LLM to determine whether this error falls under the category of attribute misalignment errors. In such cases, we utilize an automated debugging mechanism akin to AutoGen [14] to implement corrections. This process repeats multiple times until the code executes correctly or the maximum number of code correction attempts is reached. If the exception is not raised by attribute misalignment errors, we ask the LLM to determine whether the issue is due to unsupported parameters failure. The details of the handling method of such unsupported parameters failures will be explained in Section 3-D.

Successfully executing the LIC code generates a new input file. CHAT4SEED then checks whether the desired target branch is covered by this file by executing the target program. If the branch is covered, the input file is added to the seed corpus for further processing. If not, CHAT4SEED restarts the entire workflow to produce another input file until the target branch is covered or a maximum number of attempts is reached.

3.4 Binary Level Input Manipulation

As previously mentioned, real-world programs contain branches that handle exceptional corner cases (e.g., negative image widths). Although the LLM can generate the LIC without attribute misalignment errors, the external libraries called

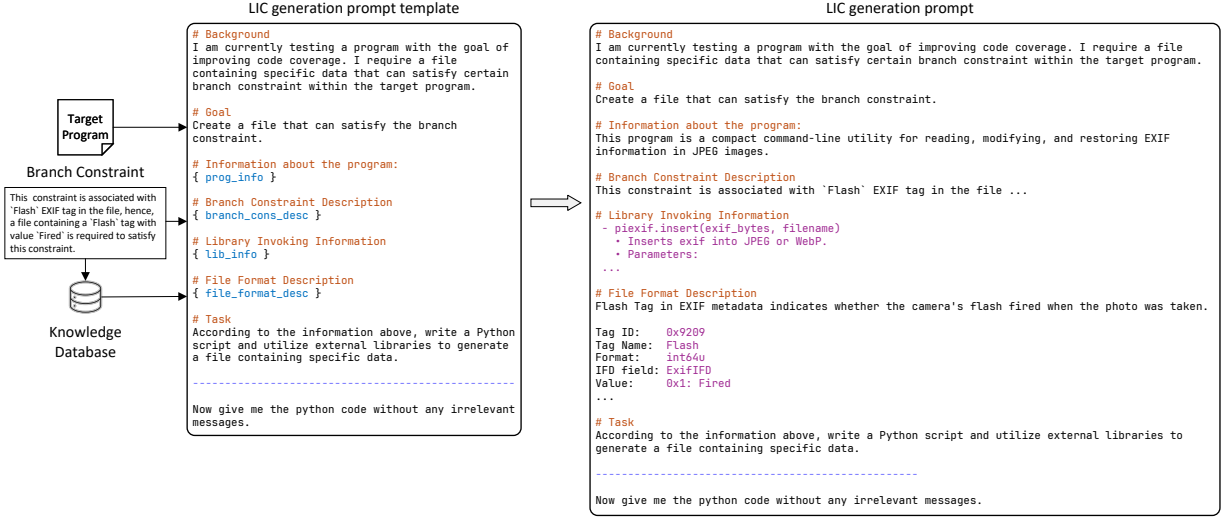


Figure 7: The LIC generation prompt construction for a given branch constraint.

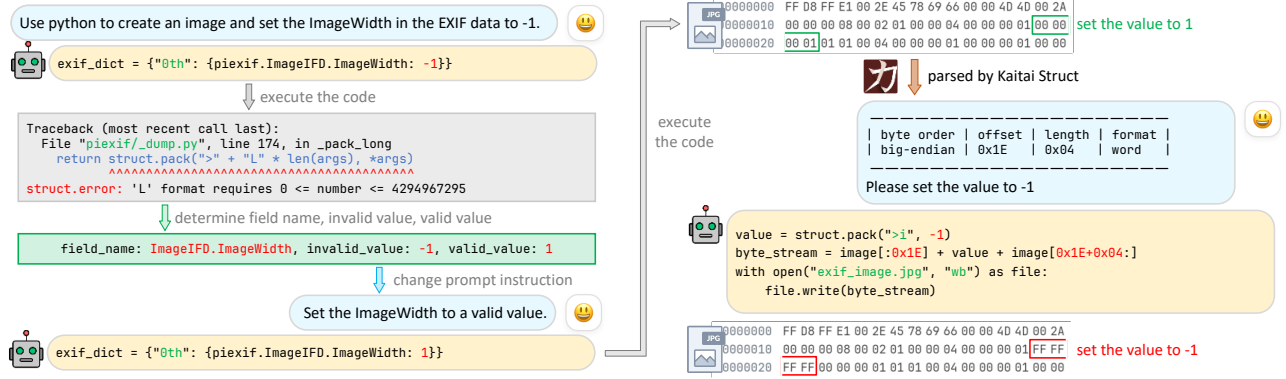


Figure 8: Unsupported parameters failure in the JPEG seeds generation. CHAT4SEED utilizes Kaitai Struct library to manipulate the data at the byte level.

by this LIC may raise errors instead of generating the actual input data, as they are not designed to handle such corner cases. For example in Fig. 8, the LLM correctly generates the Python code that sets the image width to -1 by setting the value of `piexif.ImageIFD.ImageWidth`. However, executing this code directly raises an exception because the external `piexif` library restricts the width to the range `[0, 4294967295]`.

To address the issue of unsupported field parameter errors, we propose a novel method that directly manipulates the input file at the binary level. Fig. 8 illustrates the complete process of creating a JPEG image with an invalid `ImageWidth` field. Initially, we extract the unsupported field parameter information that triggers the error. After that, we generate a valid input file containing a supported parameter using LIC prompting. We then modify the corresponding parameter at the byte level to achieve the desired unsupported parameter, thereby constructing the final input file.

a) Unsupported field parameter information extraction

Since the branch constraints may related to more than one fields of the input format, to identify which field and what parameter value in the branch constraints raise the exception,

we divide the constraints into multiple sub-constraints, each specifying a single field set to the desired value. For each sub-constraint, we generate and execute its LIC to determine whether the unsupported field parameter error persists. If the error continues to occur, we submit the error message reported by the Python interpreter, the sub-constraint, and the generated LIC to the LLM. We then request LLM to extract the field name and identify the unsupported parameter value that triggers the exception.

b) Valid file generation

After identifying the field name, we generate a valid file using the LIC with a supported parameter value. The supported parameter value for the target field is determined as follows:

- 1) If the error message specifies a range of supported values for the target field, we generate a valid value directly from that range.
- 2) In cases where the error message lacks clarity, we utilize the field name to retrieve relevant descriptions from the knowledge database and extract a valid value from them.
- 3) If the RAG does not return sufficient information, we rely on the LLM's existing knowledge to suggest a valid value.

c) Unsupported Parameter Value Embedding

To modify the value of the target field to the desired unsupported parameter value, it is essential to determine the offset, byte order, and data format of the target field at the byte level within the file generated by previous step. Kaitai Struct [15] is a powerful binary format parser that provides numerous definitions for common binary file formats. This functionality enables CHAT4SEED to execute precise binary modifications on an input file without compromising its structural integrity. Consequently, we utilize the Kaitai Struct library to identify the global offset, byte order, and data format of the target field within the input file, and then instruct the LLM to open previously created valid file in binary stream form, directly locating the target field using global offsets. Then, based on byte order, data format, and the unsupported parameter that needs to be embedded, CHAT4SEED replace the byte data of the target field, successfully achieved input file manipulation at the binary level.

4. IMPLEMENTATION & EVALUATION

We employed an LLVM compiler equipped with the SVF [16] plugin based on value flow graph to support the branch ranking module of CHAT4SEED, utilizing GLM-4 [17] as our foundational LLM for comprehending branch constraints and generating Python code.

Due to certain policy restrictions, OpenAI’s LLMs such as GPT-4o and GPT-4 are currently unavailable in our region. Instead, we use the API service provided by ZHIPU AI Foundation Model MaaS Platform to access the GLM-4 model, which ranks at the forefront of the Superbench leaderboard [18]. The default parameters we have set for GLM-4 are top-p = 0.7 and temperature = 0.5. The maxIter is set to 5 in the automatic debugging mechanism.

We evaluated CHAT4SEED to answer the following research questions:

- **RQ1:** Can CHAT4SEED generate initial seed corpus that covers more code when compared to generation-based methods and crawl-based method?
- **RQ2:** Whether the initial seed corpus generated by CHAT4SEED helps to improve the performance of off-the-shelf fuzzer?
- **RQ3:** How does the LLM perform on the task of the generation and execution of LIC, as well as binary level manipulation?

To answer the research questions mentioned above, we have selected 12 real-world programs that process various common input formats, including JPEG, TIFF, ELF, XML and XSL. Table I presents a summary of information regarding these programs.

All the experiments were conducted on an Ubuntu 22.04 LTS server equipped with 128GB of RAM and Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz*48.

4.1 Initial Code Coverage Evaluation (RQ1)

To compare with the generation-based methods, we selected Skyfire [6] and SymCC [7]. Skyfire utilizes a data-driven

TABLE I: Target program for testing.

Target	Version	Fuzzing Target	Fuzzing Options	Source Code	Input Type
jhead	3.08	jhead	"-v -autorot @@"	C	JPEG
libexif	0.6.24	fuzzer ¹	"@"	C	JPEG
exiv2	1.00.0.9	exiv2	"-v -u -pa @@"	C++	JPEG
exifprobe	2.1.0	exifprobe	"-L -a -X @@"	C	JPEG
metacam	1.2	metacam	"-a @@"	C++	JPEG
libtiff	4.6.0	tiffinfo	"-d -s -z @@"	C	TIFF
readelf	2.42	readelf	"-a @@"	C	ELF
objdump	2.42	objdump	"-a -S @@"	C	ELF
libexpat	2.5.0	xmlwf	"-k -n -p -x -w -s @@"	C	XML
libxml2	2.9.4	xmllint	"-debug -debugent -load-trace @@"	C	XML
libxslt	1.1.29	xsltproc	"-debug -load-trace -xinclude @@"	C	XSL
Sablotron	1.0.3	sabcmd	"@"	C++	XSL

¹ Compiled from *test-fuzzer-persistent.c* provided by libexif.

approach to randomly mutate and generate new seeds based on learned probabilistic context-sensitive grammar. Our evaluation was performed using its benchmark, which includes libxml2 2.9.4, libxslt 1.1.29, and Sablotron 1.0.3.

SymCC is a widely used symbolic executor that facilitates the generation of initial seed files. We initiate the engine using the sample test cases provided by AFL++ as the startup symbolic files and collect all newly generated input files as the initial seed corpus for fuzzing until the engine automatically terminates.

Additionally, we conducted a comparison with crawl-based method, using a popular open-source fuzzing seeds corpus provided by Mozilla Security [19]. This dataset has always been served as the initial corpus in other fuzzing research works[20], [21], [22], [23]. It contains initial seed files for various popular input formats, such as JPEG, TIFF and XML. The initial code coverage evaluation results for all benchmark programs are shown in Table II. CHAT4SEED outperforms the other two methods in 10 out of 12 target software programs, achieving both the highest average initial line coverage and function coverage. For instance, CHAT4SEED demonstrates the highest average line coverage at 43.4%, while Mozilla fuzzdata and SymCC cover only 22.4% and 19.8% of lines of code, respectively. Although CHAT4SEED’s code coverage in libtiff is lower than Mozilla fuzzdata, it is still significantly higher than SymCC. From the perspective of coverage per seed (i.e., coverage/count), CHAT4SEED improves the metric with 120.6% and 133.3% when compared with Mozilla fuzzdata and SymCC respectively. The reason for this performance gain is that CHAT4SEED integrates the source code of the target software and incorporates code coverage information during the seed generation process, aiming to maximize the initial code coverage. CHAT4SEED is highly tailored to the target software, with each seed having minimal information redundancy, which allows for higher initial code coverage with fewer seeds. In contrast, the Mozilla fuzzdata seed corpus is directly downloaded from the internet and contains seeds that are not specifically scoped for the target software. Additionally, SymCC encounters segmentation fault errors quickly on libexif, libtiff, objdump, and libexpat, resulting in the generation of only a few seeds and consequently leading to low code coverage.

Compared to Skyfire, in terms of line coverage, although CHAT4SEED performs slightly lower on libxml2, it achieves

TABLE II: Initial coverage details of the seed corpus generated by CHAT4SEED (listed as CHAT4.), Mozilla fuzzdata (listed as Moz.), SymCC, and Skyfile.

Program	Initial Corpus Count				Line Coverage (%)				Function Coverage (%)			
	Moz.	SymCC	Skyfile	CHAT4.	Moz.	SymCC	Skyfile	CHAT4.	Moz.	SymCC	Skyfile	CHAT4.
jhead	204	654	n/a	186	30.3	29.2	n/a	49.6	53.4	56.9	n/a	58.9
libexif	204	104	n/a	173	18.1	18.4	n/a	42.9	39.3	38.9	n/a	61.1
exiv2	204	1033	n/a	154	12.5	8.7	n/a	18.2	22.2	14.2	n/a	28.6
exifprobe	204	2783	n/a	139	8.4	19.8	n/a	34.6	17.1	32.8	n/a	57.4
metacam	204	2448	n/a	161	23.8	54.3	n/a	65.2	28.6	52.2	n/a	57.9
libtiff	84	70	n/a	140	31.1	2.9	n/a	27.4	40.5	7.0	n/a	37.2
readelf	576 ¹	4134	n/a	576	17.9	20.7	n/a	28.4	33.3	33.3	n/a	43.0
objdump	136 ¹	44	n/a	136	27.3	17.9	n/a	49.8	41.7	26.0	n/a	69.8
libexpat	1505	147	n/a	1163	42.9	10.3	n/a	51.2	42.7	17.5	n/a	46.4
libxml2	1505	1563	5923	2199	16.7	10.7	22.0	21.3	26.7	19.4	24.1	31.4
libxslt	1505	1466	5017	2088	13.5	16.3	57.4	67.2	23.8	26.0	51.9	81.0
Sablotron	1505	440	5017	1827	25.9	28.6	65.2	65.5	38.6	40.7	48.1	68.6

¹ Since Mozilla fuzzdata does not include ELF executable seeds, we leverage the `smallelf` files from the official AFL seed set and mutated them with AFL++ to generate the same number of seeds as CHAT4SEED to serve as the initial seed set.

higher coverage on libxslt and Sablotron. Additionally, CHAT4SEED surpasses Skyfire in function coverage across all programs. Notably, CHAT4SEED generates less than half the number of seeds as Skyfire. Furthermore, while Skyfire used the same XSL files as initial seeds for both libxslt and Sablotron, CHAT4SEED differentiated between these two programs and generated more targeted initial seed corpus. The data in Table 2 shows that libxml2 has a Lines of Code (LoC) count of 54.6k, making it the largest among the 12 test programs. This indicates the presence of more complex constraints, which may pose comprehension challenges for the LLM, resulting in CHAT4SEED having slightly lower line coverage on libxml2 compared to skyfire. However, for the smaller-scale programs like libxslt and Sablotron, CHAT4SEED successfully outperformed skyfire. Moreover, due to the targeted generation of seeds, CHAT4SEED still exceeded skyfire in function coverage metrics on libxml2, suggesting that CHAT4SEED’s seeds may have greater fuzzing potential.

4.2 Fuzzing Performance Evaluation (RQ2)

To evaluate whether CHAT4SEED can improve the performance of the off-the-shell-fuzzer, we chose AFL++ as the fuzzing engine and conducted experiments to determine if the seed corpus generated by CHAT4SEED helps the fuzzer achieve higher code coverage and discover more bugs during the fuzzing campaign.

For all 12 fuzzing targets listed in column 3 of Table I, since Skyfire did not disclose their generated initial seed corpus, we feed AFL++ with other 3 seed corpus (Mozilla fuzzdata, SymCC, and CHAT4SEED) listed in Table II and collect the edge coverage as well as crash numbers after 48 hours of fuzzing. Each experiment is repeated 10 times to eliminate randomness.

4.2.1 Testing Coverage

Table III lists the average edge count after 48 hours’ fuzzing. Given that Skyfire doesn’t provide the edge found by AFL in its paper, we conduct a specific comparison between CHAT4SEED, Mozilla fuzzdata, and SymCC in this section. When fed with the seed corpus generated by CHAT4SEED

TABLE III: Average edge numbers hit by AFL++ after 48 hours’ fuzzing campaign for different seed corpus.

Program	CHAT4SEED	Mozilla	Improve	SymCC	Improve
jhead	752	690	8.93%	609	23.48%
libexif	1911	1046	82.70%	1041	83.51%
exiv2	13611	10221	33.17%	8002	70.09%
exifprobe	4596	2440	88.37%	2624	75.19%
metacam	1161	618	87.97%	796	45.79%
libtiff	4278	3921	9.10%	2385	79.37%
readelf	6005	5535	8.49%	5794	3.65%
objdump	5571	5115	8.92%	5263	5.85%
libexpat	4727	4485	5.38%	3765	25.55%
libxml2	7985	7259	10.00%	4825	65.49%
libxslt	6540	6442	1.52%	6454	1.33%
Sablotron	3664	3636	0.77%	3648	0.44%
Average			28.78%		39.98%

, AFL++ achieves the highest edge count compared to the other two seed generation methods. For instance, compared to Mozilla fuzzdata, CHAT4SEED shows an improvement of up to 88.37% (exifprobe), with an average improvement of 28.78%. Compared to SymCC, CHAT4SEED shows an improvement of up to 79.37% (libtiff), with an average improvement of 39.98%. Fig. 9 presents the number of unique edges discovered by AFL++. As shown in the figure, due to higher initial code coverage on most software, CHAT4SEED maintains a significant lead in path discovery throughout the entire fuzzing process across all 12 target programs.

Based on the above data and analysis, it can be concluded that the seeds generated by CHAT4SEED have better fuzz testing potential and higher quality compared to those generated by Mozilla fuzzdata, SymCC, as well as Skyfile.

4.2.2 Crash Detection

Table IV presents the quantity of crashes discovered subsequent to fuzz testing utilizing four initial seed sets, reveal that CHAT4SEED consistently outperforms the other methods across various programs, uncovering a higher number of crashes. For instance, in the testing of exifprobe, CHAT4SEED detected 410 crashes, whereas Mozilla and SymCC identified 303 and 54 crashes, respectively. Similarly, in the testing of libxml2, CHAT4SEED excelled by discovering 382 crashes, far exceeding Mozilla’s 160 and SymCC’s 35. Additionally, CHAT4SEED demonstrated superior performance in multiple

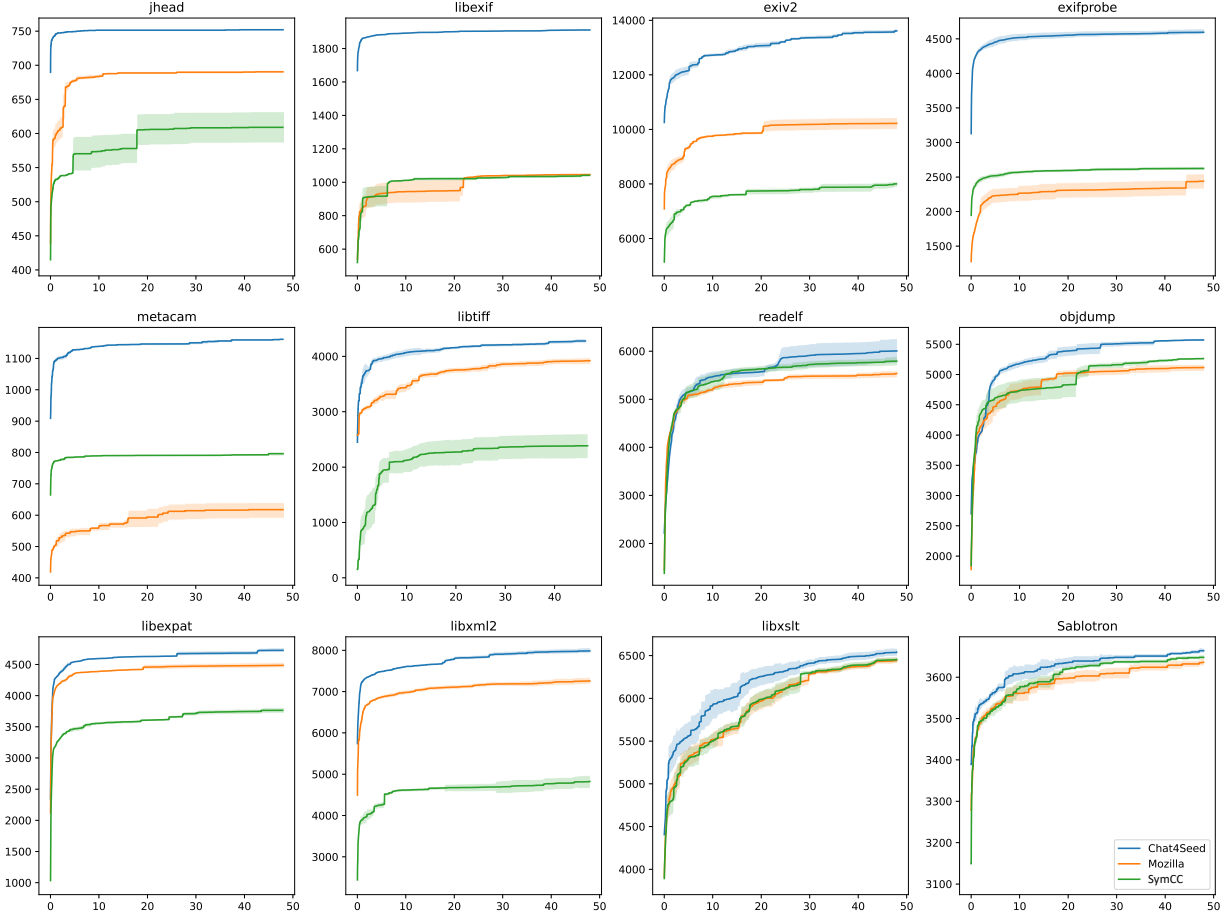


Figure 9: Unique edges discovered by AFL++. X-axis: time (bounded at 48 hours), Y-axis: the number of unique edges (bands represent 95% C.I.s).

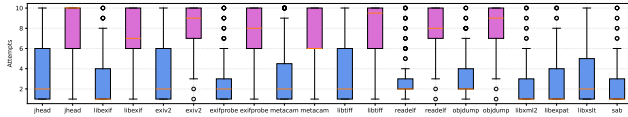


Figure 10: Branch constraint solving attempts. Blue indicates the complete CHAT4SEED workflow, while purple representation denotes the generation of binary seeds in the absence of a knowledge database (text-type seeds are not included in this comparison).

test cases, with an average crash detection quantity of 87.7, compared to 47.6 for Mozilla and 17.8 for SymCC, demonstrate that the initial seed sets generated by CHAT4SEED possess superior quality, enabling more effective guidance of fuzz testing to uncover a greater number of software vulnerabilities.

4.3 LLM performance Analysis (RQ3)

4.3.1 LIC generation analysis

Figure 10 illustrates the statistical solving attempts for each program in our benchmark focusing on the generation of functional LICs. As previously mentioned, due to LLM’s limitation of understanding constraints and the possibility of

TABLE IV: The average number of crashes discovered by AFL++ after 48 hours’ fuzzing campaign for different seed corpus.

Program	CHAT4SEED	Mozilla	SymCC	Skyfile ¹
jhead	86	6	6	n/a
libexif	0	0	0	n/a
exiv2	0	0	0	n/a
exifprobe	410	303	54	n/a
metacam	166	98	115	n/a
libtiff	0	0	0	n/a
readelf	0	0	0	n/a
objdump	0	0	0	n/a
libexpat	0	0	0	n/a
libxml2	382	160	35	18
libxslt	0	0	0	3
Sablotron	8	4	4	25
Average	87.7	47.6	17.8	n/a

¹ The crash number of Skyfire are collected after fuzzing for 15 months as listed in its paper.

execution failures in the Python code generated by the LLM, we have established a maximum of 10 attempts to solve each branch. From an overall viewpoint, the median number of solving attempts after introducing the knowledge database is concentrated around 1 and 2, the overall fluctuation of solving attempts is relatively small and stable. In contrast, this median

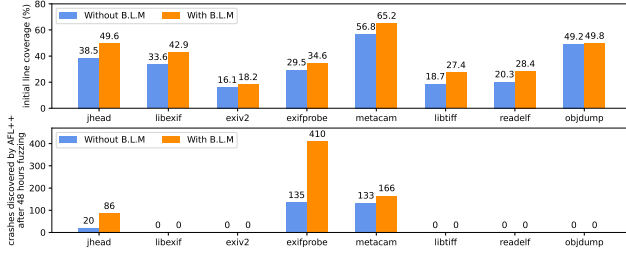


Figure 11: Initial line coverage and crashes discovered by AFL++ of seed corpus generated by CHAT4SEED with and without the binary level manipulation (listed as B.L.M).

value indicates an increase for 8 programs that take binary inputs, rising to approximately 8 attempts when the knowledge database is not utilized.

4.3.2 B.L.M analysis

We also evaluated the impact of binary level manipulation (B.L.M) on the generation of initial seeds. Fig. 11 illustrates the initial line coverage achieved by the seed corpus generated via CHAT4SEED across 8 programs that accept binary files as input, after the removal of this component. It can be observed that B.L.M resulted in an increase in initial coverage across all programs, with a specific average improvement of 23.66%. As previously mentioned, malformed input seeds are more likely to trigger program failures. Fig. 11 also illustrates the number of crashes discovered after 48 hours of fuzzing with seeds generated by CHAT4SEED, both with and without B.L.M. The experimental results can support our viewpoint that, with the application of B.L.M, the seed corpus possesses a greater potential to trigger program failures.

5. RELATED WORKS

ChatAFL [24] utilizes LLMs to extract protocol grammars for structured fuzzing, enrich initial seeds, and induce state transitions to overcome coverage plateaus. ChatFuzz [25] is a greybox fuzzing approach that integrates LLMs to enhance the generation of structured inputs. FuzzGPT [26] employs LLMs to generate edge-case programs for fuzzing Deep Learning (DL) libraries, utilizes both in-context learning and fine-tuning strategies to prime LLMs. Fuzz4All [27] employs LLMs for generating and mutating inputs across various programming languages. WhiteFox [28] is a white-box compiler fuzzer that utilizes LLMs to generate test programs targeting compiler optimizations. It features an LLM-based framework for analyzing optimization source code and producing test cases, detecting deep logic bugs in DL compilers. CovRL-Fuzz [9] combines coverage-guided reinforcement learning with LLMs to enhance bug detection and code coverage in JavaScript engines. Skyfire [6] is a data-driven approach for generating seeds for fuzzing highly-structured inputs. It employs a probabilistic context-sensitive grammar (PCSG) learned from a sample corpus and grammar, producing seeds that achieve higher code coverage and enhance bug detection rates in fuzz testing. Asmita et al. present a approach in the domain of embedded systems security by leveraging LLMs to generate

TABLE V: Comparison with related research works

Research Work	Target Programs	Research Domain ¹	Seeds Format
ChatAFL [24]	Protocols	FE	Text only
ChatFuzz [25]	Target programs that take in text-based formatted inputs	FE	Text only
FuzzGPT [26]	Deep learning libraries	FE	Text only
Fuzz4All [27]	Compiler	FE	Text only
WhiteFox [28]	Compiler	FE	Text only
CovRL-Fuzz [9]	JavaScript engine	FE	Text only
Skyfire [6]	XML & XSL parser	SG	Text only
Fuzzing Busybox [10]	Applets in busybox	SG	Text only
CHAT4SEED	Real-world programs that process various file input	SG	Both text and binary

¹ FE=Fuzzing Technique Enhancement, SG=Initial Seed Corpus Generation

initial seeds for fuzz testing, specifically targeting BusyBox [10].

Table V presents a comparison between CHAT4SEED and these related works in terms of specific research fields, the application methods of LLM, and the types of seeds processed.

6. CONCLUSION

The quality of the initial seed corpus is crucial for the efficiency of fuzz testing. In this paper, we propose a novel framework called CHAT4SEED, designed to produce a high-quality initial seed corpus. CHAT4SEED generates seeds by comprehending format-specific branch constraints in the program from a semantic perspective. By assigning each branch a weight factor, CHAT4SEED iteratively identifies the most valuable branches that have not been covered by the current seed corpus and submits them to LLMs to extract the constraints. Finally, CHAT4SEED enables LLMs to generate both text-based and binary-based seeds. It achieves this by either directly producing text files or generating functional library invocation code, which is subsequently enhanced through a knowledge database and binary-level manipulation to generate seeds in binary format. Experimental results on multiple real-world software applications demonstrate that the seeds generated by CHAT4SEED achieved the highest code coverage in fuzz testing and the greatest average number of crash discoveries, revealing the notable potential of CHAT4SEED in the domain of initial seed corpus generation.

7. DATA AVAILABILITY

Upon acceptance of this paper, we will make our work publicly available on GitHub.

REFERENCES

- [1] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, “{AFL++}: Combining incremental steps of fuzzing research,” in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, 2020.
- [2] K. Serebryany, “Continuous fuzzing with libfuzzer and addresssanitizer,” in *2016 IEEE Cybersecurity Development (SecDev)*. IEEE, 2016, pp. 157–157.

- [3] K. Serebryany, “{OSS-Fuzz}: Google’s continuous fuzzing service for open source software,” *USENIX Security Symposium ser. USENIX*, 2017.
- [4] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, “Evaluating fuzz testing,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 2123–2138.
- [5] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, and A. L. Hosking, “Seed selection for successful fuzzing,” in *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, 2021, pp. 230–243.
- [6] J. Wang, B. Chen, L. Wei, and Y. Liu, “Skyfire: Data-driven seed generation for fuzzing,” in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 579–594.
- [7] S. Poeplau and A. Francillon, “Symbolic execution with {SymCC}: Don’t interpret, compile!” in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 181–198.
- [8] L. De Moura and N. Bjørner, “Z3: An efficient smt solver,” in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.
- [9] J. Eom, S. Jeong, and T. Kwon, “Covrl: Fuzzing javascript engines with coverage-guided reinforcement learning for llm-based mutation,” *arXiv preprint arXiv:2402.12222*, 2024.
- [10] Y. Oliinyk, M. Scott, R. Tsang, C. Fang, H. Homayoun *et al.*, “Fuzzing busybox: Leveraging llm and crash reuse for embedded bug unearthing,” *arXiv preprint arXiv:2403.03897*, 2024.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [12] O. Topsakal and T. C. Akinci, “Creating large language model applications utilizing langchain: A primer on developing llm apps fast,” in *International Conference on Applied Engineering and Natural Sciences*, vol. 1, no. 1, 2023, pp. 1050–1056.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, “Autogen: Enabling next-gen llm applications via multi-agent conversation framework,” *arXiv preprint arXiv:2308.08155*, 2023.
- [15] Kaitai Project, “Kaitai struct,” 2024, [Online]. Available: <https://kaitai.io>. [Accessed: Oct. 30, 2024].
- [16] Y. Sui, D. Ye, and J. Xue, “Detecting memory leaks statically with full-sparse value-flow analysis,” *IEEE Transactions on Software Engineering*, vol. 40, no. 2, pp. 107–122, 2014.
- [17] G. Team, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai *et al.*, “Chatglm: A family of large language models from glm-130b to glm-4 all tools,” *arXiv e-prints*, pp. arXiv–2406, 2024.
- [18] Superbench Team, “Superbench for large language model,” 2024, [Online]. Available: <https://fm.ai.tsinghua.edu.cn/superbench>. [Accessed: Oct. 30, 2024].
- [19] Mozilla, “Mozilla security fuzzdata,” 2015, [Online]. Available: <https://github.com/fuzzer-io/mozilla-fuzzdata>. [Accessed: Oct. 30, 2024].
- [20] D. Wang, Y. Li, Z. Zhang, and K. Chen, “Carpetfuzz: Automatic program option constraint extraction from documentation for fuzzing,” in *Proceedings of the 32nd USENIX Conference on Security Symposium*. Anaheim, CA, USA: USENIX Association, 2023.
- [21] S. Li and Z. Su, “Finding unstable code via compiler-driven differential testing,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2023*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 238–251.
- [22] A. Fioraldi, D. C. D’Elia, and L. Querzoni, “Fuzzing binaries for memory safety errors with qasan,” in *2020 IEEE Secure Development (SecDev)*, 2020, pp. 23–30.
- [23] C. Zhang, X. Lin, Y. Li, Y. Xue, J. Xie, H. Chen, X. Ying, J. Wang, and Y. Liu, “APICraft: Fuzz driver generation for closed-source SDK libraries,” in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 2811–2828.
- [24] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, “Large language model guided protocol fuzzing,” in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.
- [25] J. Hu, Q. Zhang, and H. Yin, “Augmenting grey-box fuzzing with generative ai,” *arXiv preprint arXiv:2306.06782*, 2023.
- [26] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, “Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [27] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, “Fuzz4all: Universal fuzzing with large language models,” *Proc. IEEE/ACM ICSE*, 2024.
- [28] C. Yang, Y. Deng, R. Lu, J. Yao, J. Liu, R. Jabbarvand, and L. Zhang, “Whitefox: White-box compiler fuzzing empowered by large language models,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOP-SLA2, pp. 709–735, 2024.